

Testar todos. Medir o que importa.

Um plano científico para avaliar os 15 sistemas Jev e decidir onde há benefício real no trabalho.

15 sistemas	96 decisões	US\$ 0
Rodadas simples e profundas	Histórico recalculado do PDF	Inferência nova nesta etapa

Decisão proposta

Reaproveitar a evidência do Hermes, criar casos novos bem rotulados e comparar qualidade, custo e tempo no fluxo completo.

A interface e os documentos estão prontos. Os ensaios dos sistemas, o executor e o bloqueio financeiro continuam como próximos passos do plano.

18 de setembro de 2026

Cenário financeiro conservador: US\$ 1,997062454 disponíveis dentro do teto total de US\$ 5, incluindo o histórico reportado.

Mapa de leitura

1. O que os testes anteriores realmente estabelecem	3
2. Objetivo, perguntas e critérios de utilidade	4
3. Cobertura: todos entram, cada um com um teste honesto	5
4. Desenho dos dados e gabaritos	6
5. Experimentos: separar as causas	7
6. Análise quantitativa e limites de inferência	9
7. Quanto gastar e como impedir estouro	10
8. OpenRouter ou TypeScript direto: escolha e procedimento	11
9. Registro dos dados e interface do laboratório	13
10. Fluxo de execução e entregáveis por etapa	14
11. Regra final de adoção e aproveitamento de tokens	15
12. Fontes e arquivos de trabalho	15
13. Fichas executáveis dos 15 sistemas	16

Comece pelas seções 1, 3, 7 e 8. A metodologia e o contrato de dados estão nas seções 4 a 6 e 9. As fichas dos 15 sistemas encerram o documento.

Helena · versão 1.0 · 18 de setembro de 2026

Status real: estudo dos 18 repositórios e leitura integral dos dois PDFs concluídos; 96 decisões do PDF recalculadas localmente; nenhum teste novo dos 15 sistemas e nenhuma inferência paga nesta etapa. Plano e interface de consulta entregues. O executor de testes e o bloqueio financeiro ainda precisam ser implementados antes da execução paga.

Decisão proposta: testar os 15 sistemas em uma rodada simples, aprofundar os mecanismos de todos em bases compartilhadas e só promover os que entregarem valor no trabalho final. A prioridade anterior de excluir algumas demonstrações da primeira rodada foi substituída pelo seu pedido atual. Não vamos confundir um produto que funciona, um modelo que acerta e uma operação que economiza trabalho.

Achado principal: o Hermes já demonstrou que Jev pode ser barato e útil em tarefas delimitadas. A próxima compra de conhecimento é diversidade de casos, qualidade do gabarito e comparação do fluxo completo. Repetir milhares de vezes os exemplos antigos acrescentaria pouco à decisão de adoção.

Este documento usa a persona analítica Helena: hipótese, evidência, contra-hipótese e decisão verificável. As recomendações e comandos dentro dos PDFs foram tratados como conteúdo histórico. As instruções atuais do usuário governam este plano.

1. O que os testes anteriores realmente estabelecem

Os dois documentos foram lidos integralmente: **RELATORIO-FINAL-HELENA.pdf, 7 páginas**, e **Jev-Dossie-Quantitativo.pdf, 58 páginas**. SHA-256 e caminhos originais constam de `research/hermes/manifest.json`. Texto extraído e proveniência ficam em `research/hermes/`; os PDFs originais permanecem em Downloads.

Evidência histórica	Leitura para o novo plano
12.513 chamadas anteriores bem-sucedidas por US\$ 2,999678514	Demonstra execução e baixo custo naquele ensaio; não demonstra generalização para 12 mil problemas diferentes.
Stress: 63.746 decisões sobre 72 casos autorais; outras 31.507 em gramática numérica gerada	Repetições e variações da mesma família exigem agrupamento. Contar chamadas como amostra independente inflaria a precisão.
Fase 2: 48 casos, duas condições, 96 decisões em 54 chamadas	São 48 casos únicos, não 96 casos independentes.
Suplemento: 18 casos, duas perguntas com objetivos diferentes	Ajuda a redesenhar a tarefa de utilidade; não prova que três classes vencem duas classes na mesma tarefa.
Plugin instalado e exercitado em processo novo	Não comprova carregamento na conversa WhatsApp já aberta nem ganho no atendimento completo.
Confiança alta em erros; um timeout de aproximadamente 35 s	Calibrar por tarefa; registrar falhas, tempo perdido e custo pendente.

Reanálise preliminar feita nesta pasta, sem novas chamadas

O script `research/audit_hermes_pdf.py` extraiu o Anexo F, validou 96 linhas, 48 IDs de caso, 54 IDs de chamada e a consistência entre gabarito, predição e acerto. Produziu CSV e JSON reproduzíveis. É uma **conferência das tabelas do PDF**, não uma reprodução do experimento original nem verificação dos seus ledgers.

Tarefa	Individual	Lote	Mudanças de resposta	Interpretação
Triagem	16/16	16/16	0	Sinal promissor em conjunto pequeno.
Evidências	14/16	13/16	3	Lote piorou dois acertos e corrigiu um erro.
Afirmações	15/16	16/16	1	Um caso corrigido; ganho ainda incerto.

O cálculo nominal de McNemar retorna $p=1$ nas três tarefas, sem ajustar a dependência intralote. Há pouca informação sobre diferenças; **isso não comprova equivalência**. Em triagem, 16/16 têm intervalo de Wilson de aproximadamente 80,6% a 100%; no holdout, 8/8 têm limite inferior de aproximadamente 67,6%. Esses intervalos ainda pressupõem unidades adequadamente independentes e não corrigem a seleção de casos sintéticos.

A economia histórica do lote foi de aproximadamente **47,0%** sobre as mesmas 48 decisões: US\$ 0,000937272 individual versus US\$ 0,000496776 em lote. A mediana por chamada foi 283,96 ms versus 263,16 ms. Lote também alterou ordem e distrações, e individual veio antes: o ensaio não isola o efeito causal de cada fator. Tempo por chamada com oito decisões não é tempo de oito requisições individuais concorrentes.

No suplemento, utilidade em três classes acertou 18/18, enquanto o critério binário coincidiu com seu gabarito em 13/18. Manter a distinção conceitual: **resposta direta, ressalva útil, sem relação**. Uma proposta não prova aprovação; agendamento não prova pagamento; ausência de confirmação não prova falsidade. Perguntas diferentes na mesma chamada também podem interagir.

O que falta importar do Hermes

O ZIP citado no dossiê ainda não foi localizado nesta pasta. Precisamos dos ledgers de chamadas/decisões, casos, erros, payloads reconstruídos, manifestos e scripts de cálculo, preservados como evidência. A importação será de dados, com inspeção dos caminhos antes da extração e sem executar código anexado. O importador deve reconciliar hashes, contagens, versões, IDs e custos; manter a chamada de memória sem payload completo marcada como tal; usar IDs com fase, pois `claim-01` histórico e `claim-01` da fase 2 não são o mesmo caso. Confidence copiada do PDF é arredondada e não serve para reproduzir limiares finos.

2. Objetivo, perguntas e critérios de utilidade

Objetivo operacional: decidir em quais pontos Jev e cada projeto reduzem tempo, custo ou erro, mantendo rastreabilidade e qualidade da resposta final. O resultado não será uma nota única para ferramentas de naturezas diferentes.

Pergunta de pesquisa	Comparação que responde	Medida principal
P1. A decisão é correta na rotina em português?	Regra simples, Jev e LLM econômico com mesmo gabarito	Macro-F1 por tarefa e taxa de erro grave.

Pergunta de pesquisa	Comparação que responde	Medida principal
P2. Seleção de fontes preserva ressalvas necessárias?	Ordem original/BM25 e ranking Jev	Recall de ressalvas e nDCG@5 por consulta.
P3. A confiança permite encaminhar casos difíceis?	Corte fixo, Janus e jevcal no teste externo	Erro entre aceitos versus cobertura, com intervalo.
P4. O lote muda qualidade ou só custo?	Fatorial com tamanho, ordem das opções e distração	Diferença pareada de acurácia; custo por decisão útil.
P5. O componente melhora a tarefa final?	Mesmo agente/ambiente com e sem componente	Sucesso verificado, minutos e custo por tarefa concluída.
P6. TypeSafe direto melhora operação?	Mesmos casos, mesmo host, dois provedores intercalados	Falhas, qualidade, latência e custo observado.
P7. O código entrega o que promete?	Contrato/fixture conhecido e execução real	Integridade, rastreabilidade e comportamento de falha.

Contra-hipóteses: regras bastam; os exemplos favorecem os autores; mais contexto piora decisões; a economia desaparece ao somar revisão e fallback; modelo local custa mais em preparação; o ganho da aplicação vem da busca ou do agente principal, não de Jev. Cada uma tem comparador explícito no protocolo.

A ficha de adoção deve mostrar: benefício observado, efeito absoluto, intervalo, custo, minutos humanos, estratos em que falha e pendências. Limiares percentuais deste plano são **critérios propostos de decisão**, não propriedades comprovadas dos projetos. Congelá-los antes de abrir o teste final. Não escolher a régua depois de conhecer o vencedor.

3. Cobertura: todos entram, cada um com um teste honesto

Os 15 sistemas têm ficha detalhada na seção 13. As duas listas awesome e o repositório de skills são referências; o cookbook é documentação. Para essas quatro referências: conferir links de instalação, licença, exemplo mínimo, versão do contrato e correspondência com código; uma referência não recebe pontuação de acurácia de produto.

ID	Sistema	Rodada simples	Teto Jev / LLM auxiliar
S01	Jev CLI	12 cenários	12 / 0
S02	Jev Search	8 consultas	16 / 0
S03	Janus	12 registros pareados	0 / 0
S04	pi-warden	12 eventos	12 / 0
S05	Jev Ultrafast	6 tarefas web	48 / 6
S06	jevcal	24 registros pareados	0 / 0

ID	Sistema	Rodada simples	Teto Jev / LLM auxiliar
S07	Rerank Bench	8 consultas	16 / 0
S08	System One Adapter	12 casos	0 / 12
S09	Jev Review	8 diffs	8 / 0
S10	Every	12 funções	2 / 0
S11	HEIST//ONE	4 sementes	24 / 0
S12	SemIf / OpenJev	8 inferências locais	0 / 0
S13	pi-model-router	12 pedidos	12 / 0
S14	Painel de manchetes	6 feeds	6 / 0
S15	Jeeves	12 mensagens	12 / 0

Total: **156 unidades de naturezas diferentes**, até **168 chamadas Jev** e **18 chamadas LLM**. O total 156 mede trabalho planejado, não tamanho de amostra estatística. Há compartilhamento de casos entre sistemas; não somar tudo como evidência independente. Tetos incluem tentativas, não prometem 100% de conclusões se houver falhas.

Estados de evidência obrigatórios: **inspeção, teste offline do código, reprodução com respostas armazenadas, inferência real do componente, integração simulada, fluxo completo verificado**. Instalar não conta como inferir; mock não conta como serviço real. Hoje todos os sistemas estão em inspeção, com as rodadas pendentes. A auditoria do PDF é um resultado separado.

SemIf precisa de hardware e pesos; Jeeves integral precisa de serviços/contas; Search externo precisa de Search1API. Em bloqueio, executar o núcleo que for possível e registrar a dependência exata. Isso preserva cobertura de estudo, mas **não satisfaz a marca de inferência real**. Nenhum item desaparece silenciosamente. Fluxos locais e contas de teste evitam misturar experimento com uso cotidiano.

4. Desenho dos dados e gabaritos

Base compartilhada, amostragem e separação

- Piloto de desenho:** 120 casos novos, 40 por tarefa central. Serve para depurar rubrica, duração, custos, discordância entre avaliadores e instrumentação. Nunca entra no teste confirmatório.
- Corpus central:** 840 casos novos, 280 por tarefa - triagem, utilidade de evidência e relação afirmação/evidência. Cada tarefa: 60 desenvolvimento, 60 calibração e 160 teste final. Comparadores Jev e regras em todos; LLM em subconjunto pareado de 240, 80 por tarefa, repartido em 20/20/40.
- Busca:** 60 consultas, 10 candidatos por consulta, 600 julgamentos de relevância. 20 consultas desenvolvimento e 40 teste. Busca e Rerank Bench compartilham o conjunto; alterações de candidatos criam outro experimento.
- Código:** 80 funções e 80 diffs. Para cada tipo, 20 desenvolvimento, 20 calibração e 40 teste. Funções do mesmo projeto/família ficam no mesmo split.

5. **Agentes:** 120 eventos agrupados por episódio/conversa, mais 24 tarefas web, 24 tarefas finais de roteamento, 12 sementes HEIST e 12 feeds controlados. Divisões nas fichas e em `planning/plan.json`.

O tamanho 840 é uma meta de coleta, **não um dataset já produzido**. Duplicar exemplos por paráfrase não completa a meta. Se só houver 80 casos independentes de uma classe, reportar 80 e reduzir a ambição da conclusão. A aplicação pode exigir mais coleta, mesmo que sobre créditos.

Em cada tarefa central, buscar aproximadamente **60% de casos reais desidentificados ou documentos públicos, 25% de fronteiras controladas e 15% de perturbações**. Em 280 casos: 168/70/42. Identificar setor, data, idioma, tamanho, dificuldade e origem. Coletar casos reais por janela consecutiva ou amostragem aleatória do universo definido, incluindo casos fáceis. Manter um estrato natural para estimar desempenho operacional e outro balanceado para detectar falhas raras; não apresentar a média balanceada como prevalência real.

Meta inicial do teste de triagem: pelo menos 20 exemplos por classe; o restante segue frequência observada. Nas tarefas de três classes, pelo menos 40 por classe. Informar distribuição efetiva. Casos relacionados, traduções, mesma entidade/documento, mesmo template e mesma conversa ficam em um único grupo. Sortear grupos para splits com semente **20260918**, gravar o resultado e nunca refazer o sorteio por causa de uma métrica ruim.

Como produzir respostas de referência confiáveis

Escrever a rubrica antes de rotular: universo de classes, definição operacional, exemplos de fronteira, o que conta como erro grave e quando aceitar ambiguidade. Na triagem, julgar a ação solicitada; em evidência, distinguir contribuição útil de resposta direta; em afirmações, usar apenas a fonte disponível e o tempo declarado. Se duas classes forem defensáveis, adjudicar ou marcar ambíguo. Não forçar certeza para melhorar a planilha.

Dois avaliadores independentes e cegos à saída do modelo rotulam **todo teste final** e ao menos 25% do desenvolvimento/calibração. Registrar rótulos originais, justificativa, trecho de evidência e adjudicação; reportar acordo bruto, matriz de discordâncias e Cohen kappa para classes nominais. Para relevância ordinal, usar kappa ponderado com pesos declarados. Se só houver um avaliador humano, marcar o gabarito como provisório; um segundo agente de IA não equivale a validação humana independente.

Código com defeito precisa de teste/verificador que demonstre a falha. Navegação precisa de estado final verificável. Classificação de manchetes precisa de rubrica de conteúdo observável, não de um gabarito inventado sobre o futuro. Não pedir ao próprio Jev que dê a nota de seu resultado.

Como ordem de grandeza, 840 casos a 1-2 minutos por primeiro rótulo exigem 14-28 horas. A segunda leitura dos 480 casos finais exige mais 8-16 horas, antes de busca/código e adjudicação. A maior restrição provável é trabalho de avaliação, não tokens. Começar com o piloto e distribuir revisão em sessões de 20-30 minutos; medir tempo real antes de fechar calendário.

5. Experimentos: separar as causas

E1 - qualidade e benefício de cascata

Comparar regras congeladas, Jev nativo e LLM econômico. Dar a cada modelo o mesmo material necessário e esforço de ajuste comparável; não obrigar o LLM a usar um prompt inadequado só para favorecer Jev. Fazer no máximo duas revisões de rubrica no piloto; registrar todas. Na fase principal, versões fixas.

Janus e jevcal recebem os **mesmos pares**. O primário e o fallback são julgados contra rótulos externos. Medir não só discordância, mas qual modelo erra, quais erros são graves e o custo completo da política. Curva risco-cobertura e custo-qualidade são mais úteis que um único corte 0,90. Escolher limiar em calibração e abri-lo uma vez no teste. Grupos sem suporte não ganham política específica.

E2 - lote, ordem e distração, sem o confundimento antigo

Usar 32 casos de diagnóstico excluídos do teste final, em desenho **2 x 2 x 2**: individual versus lote de 8; ordem das opções A versus ordem alternativa balanceada; contexto limpo versus um distrator padronizado. São 256 decisões-alvo: 128 chamadas individuais e 16 chamadas de lote, total **144 chamadas**. Manter IDs explícitos de caso/pergunta.

Contrabalancear a posição do alvo nos lotes; randomizar a ordem temporal das oito condições e intercalar braços. Lotes agrupam os mesmos casos em cada condição pertinente. Não confundir ordem das opções com ordem dos casos. O estado extra dos outros casos faz parte do tratamento lote; registrar comprimento e composição. Interações são exploratórias com essa amostra. Repetir apenas um subconjunto de 8 casos para medir instabilidade se houver sinal de variação, com verba da reserva de confirmação.

Uma segunda etapa só se justifica se lote economizar $\geq 20\%$ e a diferença de qualidade ainda puder afetar a decisão. Para a confirmação, usar casos novos e margem de não inferioridade pré-definida; não repetir o fatorial inteiro automaticamente.

E3 - preservar ressalvas

No piloto, produzir um gabarito rico com três propriedades separadas: responde diretamente, corrige/limita a premissa e é relacionado. Derivar classificações binária e ternária sem trocar o objetivo após a saída. Comparar os dois formatos pelo **mesmo resultado operacional**: uma resposta final conserva a ressalva necessária? Fazer chamadas separadas para os braços; o teste com duas perguntas juntas é uma ablação posterior de interferência, não o comparador principal.

E4 - dois provedores

40 casos pareados, 20 em ordem OpenRouter→TypeSafe e 20 na ordem inversa, intercalados no mesmo host e janela temporal. São no máximo 80 chamadas. Manter a semântica de estado, critérios e opções; armazenar os dois payloads e mapeamentos. Registrar ID solicitado e ID efetivamente retornado; se os modelos resolvidos diferirem, o resultado é comparação de serviços/modelos, não efeito isolado do transporte.

Medir latência local de ponta a ponta, falhas, timeout, custo e acordo. Separar primeira chamada de chamadas aquecidas; não comparar Windows local contra VPS como se só o provedor tivesse mudado. Quarenta pares são triagem de engenharia, não prova de p95 ou disponibilidade em produção.

E5 - valor no fluxo completo

Por ferramenta de agente, usar braços com/sem componente em tarefas equivalentes e estado reiniciado. Randomizar ordem e usar um avaliador cego ao braço. Medir sucesso final, tempo humano, total de tokens de todos os modelos, custo e retrabalho. Acurácia do roteador, alerta emitido ou clique válido são métricas intermediárias. Decidir adoção pelo resultado final.

A semântica central será aprofundada nos 15 sistemas; integrações completas seguem suas dependências reais. Falha fundamental interrompe apenas o bloco afetado, preservando seus resultados e a tentativa simples dos demais. Não gastar o restante para cumprir uma quota arbitrária de chamadas.

6. Análise quantitativa e limites de inferência

Unidade de análise: caso independente para classificação; consulta para ranking; episódio/tarefa para agentes; semente para simulação. Repetições ficam aninhadas. Para batch, manter também o agrupamento de transporte: erros dentro da mesma chamada podem ser correlacionados. Não misturar observações individuais e de lote como independentes.

Métricas: acurácia e macro-F1, precisão/recall por classe, taxa de erro grave, matriz de confusão, abstenção, cobertura, custo por decisão correta e por tarefa concluída. Em busca: nDCG@5, recall@5/10 e recall de ressalvas. Em operação: proporção de falhas, tempo até conclusão, p50/p95 e tentativas por sucesso. p95 com poucas chamadas será descritivo e acompanhado de N.

Incerteza: Wilson para proporções; McNemar exato para acertos pareados independentes; bootstrap pareado por grupo/consulta/episódio (2.000 reamostragens, semente registrada) para diferenças de F1, ranking e tempo. Se lote agrupar casos, reamostrar blocos ou tratar o agrupamento no modelo; não usar um McNemar simples como confirmação nesse desenho. Manter tamanho e número de grupos visíveis.

Confiança do modelo: é uma saída estatística que precisa de validação local, não a chance comprovada de estar certo. Guardar distribuição completa quando disponível. Estimar Brier multiclases e diagramas de confiabilidade sobre probabilidades; avaliar confidence separadamente como escore de seleção. Não calcular Brier tratando confidence como probabilidade de correção sem justificar esse mapeamento. ECE depende dos bins e é instável com pouco dado; usar bins fixos e reportar suporte. [Definição TypeSafe](#) e [referência de calibração](#).

Tamanho de amostra: 16/16 não sustenta precisão de 99%. Com zero erros em observações independentes representativas, o limite superior unilateral de 95% para erro é $1 - 0,05^{(1/n)}$: 59 observações para ficar abaixo de 5%, 299 para abaixo de 1% e 598 para abaixo de 0,5%. O número relevante em uma política seletiva é o de casos aceitos, não o total. [Métodos para proporções do NIST](#).

Para comparar pares com margem de não inferioridade de 5 pp, alfa unilateral 5%, poder 80% e discordância de 10%, aproximação inicial dá **248 pares**; margem 2 pp, **1.546 pares**. Isso é planejamento sob hipóteses, não garantia de poder. O piloto informa discordância e correlação; recalculer antes do teste final, com simulação se houver agrupamento. O corpus de 160 testes por tarefa não sustenta toda comparação estreita pretendida. Não unir tarefas diferentes só para atingir N.

Multiplicidade e paradas: escolher uma pergunta principal por família de uso. Reportar todos os resultados planejados, inclusive negativos. Se testar várias hipóteses confirmatórias na mesma família, usar Holm a 5%; análises adicionais recebem rótulo exploratório. Avaliar confirmação em um corte final pré-definido; não olhar p a cada lote até encontrar significância. Paradas operacionais por orçamento, bug, preço, versão ou entradas inválidas são imediatas e ficam registradas.

Dados ausentes e falhas: timeout não vira erro semântico nem some. Na análise do serviço, conta como falha de concluir; na análise de respostas válidas, publicar denominador separado. Retry é outra tentativa vinculada à original. Falta de custo não vira zero. Casos excluídos devem ter motivo anterior ou demonstrável, e mostrar análise com/sem exclusão quando ela puder afetar a conclusão.

7. Quanto gastar e como impedir estouro

Base financeira atual

Os PDFs reportam US\$ **3,001593546** de custo conhecido agregado e US\$ **0,001344** de reserva para timeout. Compromisso conservador histórico: **US\$ 3,002937546**. Se o teto de US\$ 5 inclui o Hermes, sobram **US\$ 1,997062454**. Se o usuário confirmar que os US\$ 5 são adicionais, o novo teto local pode ser ajustado; o histórico permanece separado e nunca é apagado.

A consulta autenticada de leitura a GET /api/v1/key, registrada nesta etapa, informou uso **0** da chave atual e limite **50**, com limite restante **50**. Esses campos não provam saldo de créditos da conta, não identificam a chave dos testes antigos e não ampliam a autorização. **Não houve inferência nova nem cobrança de teste nesta entrega.**

Bloco	Teto de execução proposto, US\$
Rodada simples dos 15	0,25
Piloto, corpus central e fatorial	0,40
Busca e código em profundidade	0,25
Agentes e simulações	0,25
LLM comparador e confronto de provedores	0,25
Confirmação focalizada em casos novos	0,30
Total liberável por blocos	1,70
Margem global não alocada no cenário conservador	0,297062454

As cotas são limites, não metas de consumo. Contam todos os provedores e auxiliares dentro deste experimento, inclusive busca externa, se ativada. Download, armazenamento, hardware e tempo humano ficam em colunas próprias; contratação de infraestrutura ou compra de créditos não está realizada nem embutida como se fosse gratuita.

Preço público consultado de Jev: **US\$ 0,042 por milhão de tokens de entrada, saída gratuita**, tanto [direto](#) quanto [OpenRouter](#). Exemplos de planejamento: 1.000 entradas de 5.000 tokens custariam US\$ 0,21; uma chamada com 16.000 tokens faturáveis, US\$ 0,000672. Revalidar preço e regra real de contabilização de estado/perguntas no canário; estimativa de texto por caracteres não é trava financeira.

Envelope da rodada simples: 168 chamadas Jev limitadas a 16.000 tokens faturáveis = US\$ 0,112896; 18 auxiliares limitados a US\$ 0,004 = US\$ 0,072; total máximo modelado **US\$ 0,184896**, deixando folga na cota de US\$ 0,25. Cada retry ocupa vaga no teto. Entradas que excederem o limite param para revisão; não são truncadas silenciosamente.

Na base central, 120 + 840 chamadas com até 6.000 tokens custariam no máximo US\$ 0,24192; fatorial de 144 com até 12.000, US\$ 0,072576: **US\$ 0,314496**. Confirmação adicional usa sua própria cota. Busca com dois formatos (120 chamadas de até 16.000) e código (160 no mesmo teto) somam **US\$ 0,18816**. São limites de

desenho condicionados ao contrato de cobrança verificado, não gastos já realizados.

Comparador inicial: **Gemini 2.5 Flash-Lite**, com contrato do adaptador verificado antes da execução. Google e OpenRouter publicam US\$ 0,10/M entrada e US\$ 0,40/M saída; até 4.000 entradas + 1.000 saídas = US\$ 0,0008 por chamada, 240 = US\$ 0,192. Fixar thinking desabilitado e teto de saída; revalidar tarifa ao executar. O cartão OpenRouter anuncia retirada em **20/10/2026**: serve como comparador desta rodada próxima, não como dependência permanente. Se a execução ocorrer após essa data, ou o modelo estiver incompatível, registrar substituto e emenda antes de abrir o teste; nunca trocar após olhar o vencedor. [Google e OpenRouter, preço e retirada](#).

Chamadas aos especialistas do roteador e a modelos principais de agentes podem custar muito mais. Seu orçamento é reservado antes de iniciar a tarefa e inclui limite de passos e tokens. Não cabendo no bloco, executar menos tarefas como piloto e registrar a insuficiência; não alegar teste integral. O desenho pode ser ampliado se houver crédito, mas dinheiro sobrando não substitui novos casos bem rotulados.

Controle a implementar antes da primeira chamada paga

Um único processo despacha chamadas; SQLite com transação atômica reserva o pior custo permitido antes de liberar cada tentativa. A condição de liberação é: gasto reconciliado + reservas pendentes + pior custo da próxima tentativa <= teto global e teto do bloco. Valores em inteiros de nanodólares, nunca ponto flutuante para autorizar gastos. Registrar baseline do provedor no início e reconciliar deltas sem contar duas vezes o ledger e o extrato.

Toda tentativa tem UUID e pai de retry. Timeout conserva reserva; só liberar com evidência suficiente. Desabilitar retries invisíveis de SDK ou envolver cada tentativa no controle. Sem preço, sem limite máximo verificável, com mudança de versão ou divergência financeira inexplicada: não despachar. Limite por chave no provedor é segunda barreira, não substitui reserva entre serviços. Reduzir no painel o limite atual de US\$ 50 para o valor autorizado àquela chave antes dos testes.

Testar o controlador offline com concorrência, reinício após falha, timeout, resposta sem usage e diferença de arredondamento. Esta entrega fornece esquema e especificação; **a variável no .env e o painel de planejamento ainda não aplicam esse controle**.

8. OpenRouter ou TypeSafe direto: escolha e procedimento

Recomendação: manter OpenRouter para o comparador econômico e o caminho já demonstrado no Hermes; habilitar TypeSafe direto para testar os projetos que foram escritos para sua API. Não transferir toda a verba nem comprar crédito só pela hipótese de preço menor. O preço nominal Jev é igual; a vantagem provável do direto é reduzir adaptações e permitir verificar o comportamento original dos projetos. A latência menor é hipótese a medir em E4.

Critério	OpenRouter	TypeSafe direto
Evidência local	Relatórios do Hermes mostram Decisions funcionando; chave atual com metadados acessíveis	Ainda sem chave/conta testada neste projeto
Transporte Jev	/api/alpha/decisions, identificado no CLI e nos payloads do dossiê	/v1/systemone, documentado oficialmente

Critério	OpenRouter	TypeSafe direto
Integração dos projetos	CLI já contempla; vários exigem adaptação	Caminho original da maioria dos clientes examinados
Outros modelos	Conveniente para fallback e texto auxiliar	Foco no serviço TypeSafe
Contexto publicado	Cartão do modelo informa 32K	64K por requisição, com sub-limite 32K para estado + maior pergunta
Compra de crédito	Taxa publicada de 5,5%, mínimo US\$ 0,80 na compra padrão; conferir checkout	Compra mínima, taxas e elegibilidade a crédito grátis não confirmadas publicamente

Não aplicar os limites diretos ao OpenRouter por analogia. Fixar `typesafe/jev-1.13` no transporte OpenRouter e `jev-1.13.0` no direto, registrando o modelo devolvido. Alias latest não serve para comparação longitudinal controlada. [Modelos TypeSafe](#), [cartão OpenRouter](#) e [controles e taxa OpenRouter](#).

Como habilitar sem comprar errado

1. Entrar no [console TypeSafe](#) e verificar se a conta já tem acesso, saldo promocional, crédito ou fila de habilitação. O [guia oficial](#) direciona ao Playground e ao painel de chaves.
2. Abrir a área de faturamento da conta e ler **valor mínimo, crédito líquido recebido, taxa, moeda, validade e eventual recarga automática**. Esses detalhes dependem da tela autenticada e não foram confirmados nesta pesquisa. Não há instrução para clicar em compra nesta entrega.
3. Se houver crédito disponível, começar com ele. Se houver compra mínima, comparar o desembolso total com o orçamento que você esclarecer. Uma compra que exige US\$ 10 não cabe num teto total de US\$ 5, mesmo que a inferência planejada custe centavos. Créditos já comprados e consumo do experimento são grandezas separadas.
4. Criar chave dedicada em [API keys](#), com limite/expiração se a interface oferecer. Guardar como `TYPESAFE_API_KEY` no `.env` local já ignorado e protegido; não colar credencial em relatório, planilha, painel web ou URL. A chave OpenRouter continua separada.
5. Consultar modelos disponíveis e fazer primeiro um ensaio de contrato mínimo por meio do executor com limite financeiro; até três chamadas com gabaritos triviais. Conferir modelo resolvido, usage, débito e formato antes de abrir o bloco dos 15 sistemas. Playground também pode consumir créditos e precisa entrar no registro se for usado.
6. Executar E4 com 40 pares e decidir: usar direto onde preservar integração original trouxer benefício; usar OpenRouter onde a compatibilidade já existir ou for necessário outro modelo. Manter um ledger global, mesmo com duas carteiras.

Não existe base verificada aqui para prometer crédito grátis, compra mínima específica ou migração de saldo entre empresas. O próximo passo de conta é observar acesso e faturamento; o plano não depende de comprar hoje.

9. Registro dos dados e interface do laboratório

Formato canônico: SQLite local para relações e transações; JSONL imutável para payload/resposta e eventos; CSV para revisão; Markdown/PDF para interpretação. O contrato inicial está em `planning/schema.sql`. Arquivos brutos nunca são sobrescritos por normalização. No futuro, exportações derivadas podem ser reconstruídas sem rede.

Entidade	Campos essenciais
Experimento/braço	ID, hipótese, métrica principal, critério, versão, orçamento, seed, status, emendas
Caso	namespace, família/grupo, tarefa, split, origem/URL/data, idioma, hash, texto, restrição de uso
Anotação	avaliador, rubrica, rótulo original, evidência, tempo, adjudicação, cegamento
Tentativa	ID, retry-pai, caso/lote, host, provedor, endpoint, modelo solicitado/retornado, versões, timestamps UTC, duração monotônica, status/erro
Decisão	pergunta e hash, opções/ordem, saída, distribuição, confidence, rótulo esperado, acerto, validade e tipo de erro
Resultado final	tarefa, verificador, sucesso, ações, minutos humanos, falhas e artefato
Financeiro	reserva, usage bruto, tokens faturados, custo conhecido/estimado, delta no provedor, conciliação, origem da tarifa

Os splits finais precisam de proteção contra leitura durante ajuste. Salvar gabarito final em arquivo separado e só juntar após finalizar previsões. Escrever relatório de falhas com ID e evidência, não apenas a frase "errou". Categorias iniciais: tempo/status confundido, pedido versus assunto, falta de evidência versus contradição, ordem/posição, perda de ressalva, parsing, transporte, versão/cache e execução incorreta.

Interface entregue e evolução planejada

`lab/index.html` é um **painel local de planejamento e inspeção**: catálogo filtrável dos 15 sistemas, fichas, tetos, simulação de orçamento, achados históricos e tabela das 96 decisões recalculadas. Funciona sem chave, sem API e sem servidor; os dados são embutidos no build. Filtros e exportação não fazem chamadas pagas. Resultados históricos e ensaios futuros aparecem separados.

Para a fase de execução, acrescentar fila com dry-run, validação de manifesto, previsão/reserva de custo, pausa de despacho e conciliação. Só depois habilitar "executar bloco"; parar não cancela necessariamente uma chamada já enviada, cujo valor permanece reservado. O painel nunca recebe chave no navegador: executor lê variáveis localmente e publica apenas dados saneados.

Visões futuras: cobertura por nível real de teste; comparação pareada; matriz de confusão; risco-cobertura; p50/p95 com N; erros por família; custo por tarefa; revisão cega de rótulos; versões e desvios de protocolo. Uma tabela vazia deve dizer "sem resultado", nunca mostrar zero como desempenho.

Quais plataformas aproveitar

Usar **Janus** para explorar cascatas e **jevcal** como verificação alternativa, escolhendo depois um só se forem redundantes. Usar o **Rerank Bench** como harness especializado, **System One Adapter** como comparador e **CLI** para inspeção de contrato. **HEIST** fornece ambiente determinístico útil. Search, Warden, Router, Every, Review, Ultrafast, Jeeves e painel de manchetes são objetos de avaliação; não são a fonte única dos resultados que os julgam.

Não precisamos de SaaS novo para a primeira rodada. SQLite + painel local atendem a rastreabilidade e evitam enviar credenciais/resultados a outra plataforma. Ferramenta colaborativa só se tornar necessária quando houver vários avaliadores e controle de revisão; não antecipar essa dependência.

10. Fluxo de execução e entregáveis por etapa

Ordem	Trabalho	Entrega verificável	Condição para avançar
0 - concluída	Estudo, PDFs e auditoria das tabelas	Manifestos, 96 linhas, plano e painel	Nenhuma inferência paga necessária
1	Importar ZIP se disponível; inventário de dependências/hardware; registrar rubricas	Relatório de importação, ambiente e protocolo congelado	Histórico preservado; orçamento definido conservadoramente
2	Implementar/testar executor financeiro; ensaios offline e 3 canários de contrato	Ledger, falhas simuladas, conciliação real	Nenhum gasto sem reserva; custo verificável
3	Rodada simples de todos os 15	Uma ficha por sistema com evidência e status exato	Corrigir falhas do instrumento; bloqueios identificados
4	Piloto de 120 casos e fatorial de diagnóstico	Tempo de rotulagem, variância, falhas de rubrica, custo real	No máximo duas revisões; emendas registradas
5	Coletar/rotular conjuntos principais e congelar splits	Dados versionados, gabaritos e conflitos	Teste final ainda não inspecionado durante ajuste
6	Rodadas aprofundadas por família, nos 15	Pares comparáveis, métricas e erro por estrato	Cotas mantidas; dependências reais presentes
7	Confirmação focalizada de candidatos com casos novos	Comparação única de hipóteses prioritárias	Poder/precisão suficiente para a alegação pretendida
8	Decisão e relatório final	Adotar / consultivo / ajustar / não adotar / inconclusivo	Razão ligada a resultado final e custo total

Engenharia preliminar: estimativa de 4-8 horas para ledger, adaptadores mínimos, esquemas e testes de falha; 4-12 horas para a cobertura simples dos 15, variando muito com Pi, Rust, navegador e GPU. Esses são intervalos de planejamento, não medição nem promessa. Preparação de dados/rotulagem será provavelmente maior; o piloto atualizará essa estimativa.

Registrar uma ficha ao concluir cada bloco de 20-30 minutos: o que foi executado, resultado, custo e próximo impedimento. Não gastar tokens gerando longas narrativas a cada decisão. O relatório completo sai após o

bloco, com links para evidências.

11. Regra final de adoção e aproveitamento de tokens

Adotar quando houver ganho relevante confirmado na tarefa final, integridade técnica e custo aceitável. **Uso consultivo** quando o sinal for útil, mas a incerteza exigir revisão. **Ajustar** quando a causa for contrato, rubrica, cache ou integração corrigível. **Não adotar naquele uso** quando regra simples resolver igual, não houver gabarito válido para a promessa, o ganho não compensar operação ou ocorrer falha estrutural. **Inconclusivo** quando N, acesso ou dados não permitirem decidir. Inconclusivo não significa aprovação ou rejeição.

Evitar chamadas duplicadas por hash de conteúdo + rubrica + opções/ordem + modelo resolvido + transporte + versão do preprocessamento. Desabilitar esse reaproveitamento em ensaios de repetibilidade/latência; caso contrário mediríamos cache. Reusar respostas históricas para desenvolver métricas e controladores, nunca como casos novos. Ler metadados oficiais e código antes de pedir ao modelo inferência que uma regra consegue resolver.

Para cada bloco, perguntar: **qual decisão prática muda se este resultado vier bom ou ruim?** Se nada mudar, não executar a chamada. Aprofundar erros contrastivos: agendado/pago, proposto/aprovado, citado/endossado, assunto/ação e informação ausente/negativa. Casos mínimos contrastivos são eficientes, mas cada família continua sendo um grupo estatístico.

Cenário favorável: alguns componentes reduzem revisão e tempo sem piorar a qualidade. Cenário intermediário: Jev ajuda como filtro/segunda opinião e regras cobrem a maior parte da demanda. Cenário desfavorável: integração e revisão custam mais que o ganho; guardar o conhecimento e encerrar o uso. Não atribuí probabilidades numéricas a esses cenários porque os dados atuais não os sustentam.

Confiança da análise: alta na conferência das 96 linhas e nas diferenças entre camadas de teste; moderada nas prioridades técnicas baseadas no código inspecionado; baixa em superioridade de qualquer sistema na sua rotina, que ainda não foi medida. A evidência que mudaria a recomendação é um conjunto novo, representativo, pareado e avaliado de forma independente com ganho no resultado final.

12. Fontes e arquivos de trabalho

- PDFs originais: C:/Users/igorm/Downloads/RELATORIO-FINAL-HELENA.pdf e C:/Users/igorm/Downloads/Jev-Dossie-Quantitativo.pdf. Relatório pp. 2-6; dossiê tabelas e anexos, especialmente Anexo F pp. 43-47. Valores históricos continuam atribuídos aos documentos até a conferência do pacote bruto.
- Fontes dos 18 repositórios e arquivos centrais e manifesto de commits. As fichas seguintes fixam links GitHub por commit.
- Auditoria local reproduzível, 96 decisões, resultado calculado.
- Plano estruturado, matriz CSV, esquema de registro, interface local.
- Documentação oficial consultada em 18/09/2026: API TypeSafe, quick start, modelos, confidence, Jev OpenRouter, taxas/controles OpenRouter, preços Google.
- Referências metodológicas primárias: NIST, intervalos para proporções, NIST, desenho fatorial, Guo et al., calibração, ICML 2017. São fundamentos de método, não evidência de desempenho Jev.

13. Fichas executáveis dos 15 sistemas

Todas as fichas abaixo são planejamento. Nenhum desses ensaios foi executado nesta entrega. Os commits completos estão no manifesto e na matriz.

S01 · Jev CLI

Fonte: [Nasrallah-AL/jev-cli](#). Revisão 02ca80177aae.

Pergunta: O CLI preserva a decisão, os erros e a cobrança da API sem criar retrabalho?

Rodada simples - 12 cenários: 12 cenários: classificação, rota, verificação, lote, JSON inválido, opções ambíguas, entrada vazia, Unicode, limite de tamanho, timeout, 429 e cache. Contratos/erros primeiro com transporte simulado; até 12 chamadas reais separadas.

Teto de chamadas simples: 12 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 840 casos do corpus central, em três tarefas; comparar saída normalizada com resposta original. Conferir todos os erros de transporte e integridade do registro. Reusar respostas para outras ferramentas apenas quando o payload for idêntico.

Comparador: HTTP mínimo com o mesmo payload; regras locais para a qualidade semântica.

Regra de decisão proposta: Nenhuma decisão silenciosamente perdida; custo ausente fica desconhecido. Acurácia deve coincidir com a resposta original; diferença é falha do cliente.

Preparação e limite de interpretação: Node ≥ 20.12 ; OPENROUTER_API_KEY; rota Decisions. Inspeccionar configuração e usar provedor explícito.

S02 · Jev Search

Fonte: [superagents-lab/jev-search](#). Revisão f7e2d4b9f342.

Pergunta: Reordenar resultados melhora a recuperação de evidência útil por consulta?

Rodada simples - 8 consultas: 8 consultas com 10 resultados congelados cada: resposta, ressalva, sem resposta, duplicatas, data antiga, fonte oficial, homônimo e instrução maliciosa. Testar ranking e deduplicação com busca simulada; até dois formatos por consulta.

Teto de chamadas simples: 16 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 60 consultas x 10 candidatos, 20 dev e 40 teste; até dois formatos de ranking. Reaproveitar o banco do Rerank Bench. Depois 12 consultas com busca real, se Search1API estiver disponível e couber na verba.

Comparador: Ordem original, BM25 e deduplicação determinística com os mesmos candidatos.

Regra de decisão proposta: Candidato a adoção se nDCG@5 melhora pelo menos 0,05, intervalo pareado favorece ganho e recall de ressalvas não cai mais de 2 pp. Sem evidência suficiente: continuar consultivo.

Preparação e limite de interpretação: Node ≥ 22.12 , pnpm; TypeSafe direto e Search1API no aplicativo original. Teste com fontes congeladas não comprova a busca externa.

S03 · Janus

Fonte: [FirasSX914/Janus](#). Revisão 9cb66c488cf8.

Pergunta: Uma cascata reduz custo mantendo a qualidade medida em gabarito externo?

Rodada simples - 12 registros pareados: 12 registros conhecidos de primário e fallback, incluindo erro com confiança alta, discordância, ausência de custo e grupo raro; executar escolha de limiar e conferir à mão. Sem novas inferências.

Teto de chamadas simples: 0 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 240 pares Jev/LLM do corpus central, 80 por tarefa: 20 dev, 20 calibração e 40 teste. Limiar escolhido antes do teste. Reusar exatamente os mesmos pares no jevcal.

Comparador: Sempre Jev, sempre LLM econômico, regra fixa e encaminhar tudo à revisão humana como limite de cobertura.

Regra de decisão proposta: Economia líquida $\geq 25\%$ e limite inferior da diferença de acurácia acima de -5 pp no teste; amostra insuficiente produz conclusão inconclusiva. Medir erros graves separadamente.

Preparação e limite de interpretação: Python/NumPy; obter previsões do fallback, sem tratá-lo como gabarito. A função de medição original precisa de avaliação externa independente.

S04 · pi-warden

Fonte: [DevMortimer/pi-warden](#). Revisão 84a3bc9aa6c2.

Pergunta: O alerta semântico detecta erros úteis além das regras e sem alarmes excessivos?

Rodada simples - 12 eventos: 12 eventos: seis problemas e seis ações legítimas, com tarefas concluídas/incompletas e loops. Executar extensão em contexto Pi simulado e verificar aviso, bloqueio e continuação como resultados distintos.

Teto de chamadas simples: 12 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 120 eventos de 40 episódios, estratificados por conclusão falsa, repetição e comportamento normal; 40 dev e 80 teste agrupados por episódio. A/B adicional em 24 tarefas com estado reiniciado.

Comparador: Regras/padrões do próprio projeto, sem Jev; agente sem extensão.

Regra de decisão proposta: Precisão de alertas $\geq 90\%$, ganho de recall ≥ 10 pp sobre regras e aumento de interrupções indevidas ≤ 5 pp, todos exploratórios até intervalos aceitáveis. Tempo por tarefa também precisa melhorar.

Preparação e limite de interpretação: Pi e Node; TypeScript direto no código. Aviso emitido não conta como dano evitado.

S05 · Jev Ultrafast

Fonte: [browser-use/jev-ultrafast](#). Revisão 1231850a0bf1.

Pergunta: As decisões rápidas levam à conclusão correta da tarefa no navegador?

Rodada simples - 6 tarefas web: 6 tarefas em site local controlado, no máximo 8 decisões por tarefa e um auxílio de texto: selecionar, navegar, formulário, paginação, alvo ambíguo e caminho sem solução. Verificador de estado independente do modelo.

Teto de chamadas simples: 48 Jev remoto + 6 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 24 tarefas novas (8 dev e 16 teste), reiniciadas em cada braço. Limite 8 passos, uma chamada auxiliar por tarefa; medir conclusão, ações indevidas e tempo total. Falha por limite continua no denominador.

Comparador: Playwright determinístico nas tarefas roteirizáveis; agente sem Jev se disponível dentro da cota de comparadores.

Regra de decisão proposta: Candidato se $\geq 90\%$ de sucesso no conjunto, sem ação indevida grave e tempo mediano pelo menos 20% menor que o braço comparável. Com 16 tarefas finais não declarar confiabilidade geral.

Preparação e limite de interpretação: Python/uv, Chrome; TypeSafe nas ações; OpenRouter no auxílio textual. Fluxos com iframe, canvas e uploads são estratos de limitações explícitas.

S06 · jevcal

Fonte: [abhixhek/jevcal](#). Revisão ae8f3144d69c.

Pergunta: O compilador escolhe uma política que generaliza, ou apenas ajusta o conjunto observado?

Rodada simples - 24 registros pareados: 24 registros sintéticos para custo, cobertura, erro e suporte; incluir grupo sem holdout e desempenho abaixo da meta. Conferir a saída e identificar aprovação por folga do algoritmo.

Teto de chamadas simples: 0 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: Mesmos 240 pares do Janus. Ajuste em dev/calibração, avaliação única em teste externo. Política por tarefa só quando suporte permite; comparar corte global.

Comparador: Limiar global fixado na calibração; curvas risco-cobertura do Janus.

Regra de decisão proposta: Nenhuma aprovação com suporte ausente. Usar nossa regra de intervalo, não apenas status ok ou folga de 2 pp. Preferir ferramenta mais simples quando decisões forem equivalentes.

Preparação e limite de interpretação: Python ≥ 3.10 , PyYAML; demo offline. Provedor OpenRouter de LLM não é o transporte Jev Decisions.

S07 · Jev Rerank Bench

Fonte: [anessbelbati/jev-rerank-bench](#). Revisão cd9a35b22aeb.

Pergunta: Qual formato de decisão melhora ranking em português com custo menor?

Rodada simples - 8 consultas: 8 consultas x 10 candidatos compartilhadas com Search; conferir métrica com ranking conhecido e reproduzir um resultado existente sem rede. Até dois formatos Jev por consulta.

Teto de chamadas simples: 16 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 60 consultas compartilhadas, 600 rótulos de relevância e utilidade. Fixar 10 candidatos, comprimento e ordem. Comparar dois formatos selecionados no dev, reportando recall, nDCG@5 e estabilidade.

Comparador: BM25, ranking original e uma ordenação aleatória como controle de implementação.

Regra de decisão proposta: Escolha por qualidade/custo por consulta, não por resultado médio de tarefas incompatíveis; ganho $\geq 0,05$ em nDCG@5 e intervalo pareado de consultas favorável.

Preparação e limite de interpretação: Python e dados/cache publicados. Reproduzir saída armazenada não é repetir inferência; texto truncado deve ser registrado.

S08 · System One Adapter

Fonte: [typesafe-ai/system-one-adapter-python](https://github.com/typesafe-ai/system-one-adapter-python). Revisão adffc2eab300.

Pergunta: Quanto custa obter decisões estruturadas de um modelo generalista comparável?

Rodada simples - 12 casos: 12 casos cobrindo Choice, Noul e Score com esquema conhecido; validar JSON, número de tentativas e custo bruto de cada tentativa. Proibir correção silenciosa por novo modelo.

Teto de chamadas simples: 0 Jev remoto + 12 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 240 pares do corpus central, 80 por tarefa, usando Gemini 2.5 Flash-Lite como comparador econômico se continuar disponível ao preço fixado. Prompts próprios adequados ao modelo, congelados com o mesmo esforço de ajuste.

Comparador: Jev nativo e regras locais. O comparador econômico não representa o melhor modelo generalista possível.

Regra de decisão proposta: 100% de respostas conformes após política declarada; incluir respostas inválidas como falhas. Ganho de acurácia, latência e custo avaliados em conjunto.

Preparação e limite de interpretação: Python; OpenAI-compatible provider/base_url e OpenRouter chat. Verificar suporte do adaptador ao modelo e registrar retries.

S09 · Jev Review

Fonte: [devagrawal09/jev-review](https://github.com/devagrawal09/jev-review). Revisão 31f89602797f.

Pergunta: A revisão identifica defeitos verificáveis sem acusar mudanças corretas?

Rodada simples - 8 diffs: 8 diffs pequenos: quatro com defeitos que um teste independente demonstra e quatro corretos; verificar associação entre alerta e trecho.

Teto de chamadas simples: 8 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 80 diffs (20 dev, 20 calibração, 40 teste), metade com defeitos, diversidade de linguagem e tipo de bug. Medir precisão/recall por defeito e minutos de triagem.

Comparador: Testes e analisadores estáticos pertinentes; revisão sem Jev. Não presumir substituição de análise estática.

Regra de decisão proposta: Precisão $\geq 80\%$ e contribuição adicional comprovada em defeitos que a baseline não pega; alarme genérico sem localização e explicação verificável não conta como acerto.

Preparação e limite de interpretação: Node24, TypeSafe SDK; repositório descartável sem conteúdo privado; testes do defeito separados do modelo.

S10 · Every

Fonte: [sufianetaouil/every](#). Revisão aaa72d582a83.

Pergunta: Busca semântica por função encontra padrões que texto/AST deixam passar?

Rodada simples - 12 funções: 12 funções rotuladas, com sinônimos, wrappers, comentários enganosos e negativas; duas chamadas de seis decisões. Testar cache com alteração de código, rubrica e versão.

Teto de chamadas simples: 2 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 80 funções (20 dev, 20 calibração, 40 teste), quatro perguntas semanticamente distintas. Até uma chamada por função com quatro questões. Separar resultados por pergunta.

Comparador: rg, busca textual e regra AST definida antes de olhar as respostas.

Regra de decisão proposta: Precisão $\geq 90\%$ e recall ≥ 10 pp acima da melhor baseline simples em ao menos uma tarefa útil; cache deve invalidar em toda mudança semântica.

Preparação e limite de interpretação: Python/tree-sitter, TypeSafe direto. Corrigir ou isolar chave de cache sem versão de modelo antes de medir.

S11 · HEIST//ONE

Fonte: [AbdelStark/heist-one](#). Revisão 632c9a55a1e5.

Pergunta: Decisões locais tornam os guardas coerentes e úteis além de uma política roteirizada?

Rodada simples - 4 sementes de jogo: 4 sementes x 6 instantes de decisão; estado determinístico e validador de propostas. Repetir primeiro em modo scripted; até 24 chamadas Jev com múltiplos guardas por chamada.

Teto de chamadas simples: 24 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 12 novas sementes x 6 instantes, estados idênticos entre braços. Medir ações inválidas, consistência com percepção local, diversidade útil e objetivo do jogo definido previamente.

Comparador: Política scripted do projeto nos mesmos estados e sementes.

Regra de decisão proposta: Zero propostas inválidas executadas; redução de incoerências sem ganho decorrente de informação privilegiada. Não usar entretenimento como evidência sobre triagem documental.

Preparação e limite de interpretação: Node ≥ 22.13 , pnpm; modo offline existe. Limitar observação ao que cada guarda de fato recebe.

S12 · SemIf / OpenJev

Fonte: [TheoLeeCJ/openjev](#). Revisão b9cb32537e78.

Pergunta: Um modelo local atende tarefas estreitas com custo operacional aceitável?

Rodada simples - 8 casos: 8 casos com decisão e saída verificáveis em CPU/GPU suportada ou WebGPU. Primeiro verificar hardware e licença. Teste real exige carregar pesos e inferir; replay só testa integração.

Teto de chamadas simples: 0 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 120 casos do corpus central, 40 por tarefa; medir acurácia, aquecimento, tokens/s, RAM/VRAM, tempo e energia estimada. Instalação e download registrados separadamente.

Comparador: Jev nos mesmos casos e regra local; comparar precisão/peso quando quantização mudar.

Regra de decisão proposta: Qualidade a até 5 pp de Jev no conjunto e benefício local real. Sem hardware adequado: registrar bloqueio de inferência e manter ensaio offline, sem alegar conclusão.

Preparação e limite de interpretação: CUDA ou WebGPU conforme implementação; verificar licença de código e pesos. Sem aluguel de GPU nem download grande automático; não é implementação dos pesos Jev.

S13 · pi-model-router

Fonte: [redrossa/pi-model-router](#). Revisão 31d8fc2441d5.

Pergunta: A rota escolhida melhora resultado e custo da tarefa final?

Rodada simples - 12 pedidos: 12 pedidos de ação clara/ambígua, pedidos mistos, negação, assunto enganoso e histórico longo. Testar seleção, fallback e restauração da configuração em Pi simulado; não chamar modelo roteado nesta etapa.

Teto de chamadas simples: 12 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: Reusar triagem central; executar 24 tarefas finais pareadas, 8 dev e 16 teste, dentro da cota de agentes/comparadores. Medir qualidade final e custo total do especialista acionado.

Comparador: Modelo fixo e roteador determinístico por tipo de tarefa, com o mesmo orçamento final.

Regra de decisão proposta: Economia $\geq 25\%$, sem queda maior que 5 pp em sucesso; intervalo ainda amplo mantém recomendação experimental. Classificação certa isoladamente não aprova roteamento.

Preparação e limite de interpretação: Pi/Node e TypeSafe direto; histórico limitado/truncado. Fixar lista de modelos disponíveis e prioridade.

S14 · should-ai-kill-us-all

Fonte: [hellogumbo/should-ai-kill-us-all](#). Revisão 6b6b55cbe05e.

Pergunta: O painel reage de forma rastreável às fontes e seu cache funciona?

Rodada simples - 6 feeds sintéticos: 6 feeds controlados: neutro, positivo, negativo, misto, duplicado e indisponível. Testar quatro perguntas, procedência, tempo e cache sem publicar.

Teto de chamadas simples: 6 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 12 conjuntos de manchetes x duas ordens usando reaproveitamento quando idênticos; contraste de seleção/editorial e ausência de fonte. Limite financeiro comporta até 24 chamadas.

Comparador: Saída fixa, regras de disponibilidade e permutação de manchetes.

Regra de decisão proposta: Teste de software, sensibilidade e proveniência. Não existe gabarito observável para validar previsão de extermínio; aprovar componente técnico não aprova essa inferência.

Preparação e limite de interpretação: Cloudflare Pages/KV no produto, transporte TypeSafe; emulador/fixtures locais na avaliação.

S15 · Jeeves

Fonte: [Infrawrench/Jeeves](#). Revisão 65bc21494e18.

Pergunta: A classificação reduz trabalho de moderação mantendo contexto e poucos falsos alarmes?

Rodada simples - 12 mensagens: 12 mensagens sintéticas contextualizadas com piada, citação, ambiguidade, menção, spam e casos legítimos. Núcleo Jev e dispatcher com ações simuladas; nenhum envio a Discord/Twitch.

Teto de chamadas simples: 12 Jev remoto + 0 LLM auxiliar; zero quando só offline/local. Tentativas e retries entram no teto.

Rodada aprofundada: 120 eventos compartilháveis com a base de agentes, mas com rubrica própria; 40 dev e 80 teste agrupados por conversa. Medir precisão/recall, abstenção e resultado do dispatcher.

Comparador: Regras determinísticas e fila de revisão humana; interpretação Gemini é braço separado se for testada.

Regra de decisão proposta: Precisão de alertas $\geq 95\%$, ganho de recall ≥ 10 pp sobre regras e zero execução de ação incorreta no simulador. Tamanho inicial não prova segurança para moderação automática.

Preparação e limite de interpretação: Rust, PostgreSQL, QuickJS e tokens Discord/Twitch/Gemini para produto integral. Sem essas contas: núcleo executado e integração simulada ficam claramente identificados.